

Compiler Design

Parsing Concepts — Exam-Ready Notes

1. Basic Parsing Approaches

Parsing is the process of checking whether a string of tokens generated by the lexical analyzer follows the grammar rules of the language. It constructs a **parse tree** for the given input program.

There are **two main parsing approaches**:

- **Top-Down Parsing**
- **Bottom-Up Parsing**

2. Top-Down vs Bottom-Up Parsing

Feature	Top-Down Parsing	Bottom-Up Parsing
Parse tree construction	From root to leaves	From leaves to root
Starting point	Start symbol	Input string
Derivation used	Leftmost derivation	Rightmost derivation (in reverse)
Complexity	Simpler	More complex
Examples	Recursive descent, Predictive parser	Shift-reduce, LR parser

Top-down parsing tries to **derive the input string from the start symbol** by repeatedly expanding non-terminals.

3. Top-Down Parsing

Top-down parsing constructs the **parse tree from the root to the leaves**. The parser starts with the **start symbol** and tries to transform it into the **input string** using grammar rules.

Key Idea: At every step, the **leftmost non-terminal is expanded**. So top-down parsing attempts to find the **leftmost derivation** of the input string.

4. Working Mechanism of Top-Down Parsing

1. Start with **start symbol S**
2. Compare generated symbols with input tokens
3. Expand non-terminals using production rules
4. Continue until input matches grammar → **success**, or mismatch occurs → **error**

Top-down parsing works like **predicting which rule to use based on input**.

5. Recursive Descent Parsing

Recursive descent parsing is the **most general form of top-down parsing**. Each **non-terminal in the grammar is implemented as a recursive function**.

Example grammar:

```
S → cAd
A → ab | a
```

Input string: **cad**

Parsing Process:

- Start with **S**
- Expand using rule $S \rightarrow cAd$
- Match **c** with input
- Expand **A**, try first rule $A \rightarrow ab$
- If mismatch occurs → **backtrack**
- Try next rule $A \rightarrow a$
- Input matches → **parsing successful**

6. Implementation Using Recursive Procedures

Each nonterminal is represented as a **procedure in the parser**:

```
procedure S() {  
    match('c');  
    A();  
    match('d');  
}  
  
procedure A() {  
    if(next token == 'a')  
        match('a');  
}
```

7. Advantages of Recursive Descent Parser

- ✓ Simple to understand and implement
- ✓ Easy to write manually
- ✓ Suitable for small grammars
- ✓ Clear structure (each rule → function)

8. Limitations of Recursive Descent Parser

- ✗ May require **backtracking**
- ✗ Cannot handle **left recursive grammars**
- ✗ Inefficient if grammar is ambiguous
- ✗ Multiple scans of input may occur

Because of these problems, **predictive parsing** is preferred.

9. Predictive Parsing

Predictive parsing is a **special type of recursive descent parsing** that **does not require backtracking**. The parser can **predict which production rule to use** by looking at the next input symbol.

To make predictive parsing possible, the grammar must:

1. **Remove left recursion**
2. **Apply left factoring**

10. Idea of Lookahead

Predictive parsing uses **lookahead symbols** — the parser **looks at the next input token** before choosing a production rule.

Example:
 $E \rightarrow T E'$
 $E' \rightarrow + T E' \mid \epsilon$

- If next token is **+** $\rightarrow$ choose $E' \rightarrow + T E'$
- If next token is **end** $\rightarrow$ choose $E' \rightarrow \epsilon$

11. Table-Driven Parsing

Predictive parsing can also be implemented using a **parsing table**. Components required:

1. **Input Buffer**
2. **Stack**
3. **Parsing Table** — written as $M[A, a]$ where A = nonterminal, a = terminal symbol

12. LL(1) Grammars

LL(1) grammars can be parsed using **predictive parsing without backtracking**.

Symbol	Meaning
First L	Scan input Left to right
Second L	Produce Leftmost derivation
1	Use 1 lookahead symbol

A grammar is LL(1) if its **parsing table has no multiple entries**. It must be unambiguous and not left recursive.

13. FIRST and FOLLOW Sets

These two functions are used to **construct predictive parsing tables**.

FIRST Set

$FIRST(\alpha)$ is the set of **terminals that appear first in strings derived from α** .

- If X is a terminal $\rightarrow FIRST(X) = \{X\}$
- If $X \rightarrow \epsilon \rightarrow$ add ϵ to $FIRST(X)$
- If $X \rightarrow Y_1 Y_2 Y_3 \rightarrow$ add $FIRST(Y_1)$

Example: $F \rightarrow (E) \mid id \rightarrow FIRST(F) = \{ (, id \}$

FOLLOW Set

$FOLLOW(A)$ is the set of **terminals that can appear immediately after A** in some production.

- Add **\$** to $FOLLOW(\text{start symbol})$
- If $A \rightarrow \alpha B \beta \rightarrow$ add $FIRST(\beta) - \epsilon$ to $FOLLOW(B)$
- If $A \rightarrow \alpha B \rightarrow$ add $FOLLOW(A)$ to $FOLLOW(B)$

14. Building Predictive Parsing Tables

Steps to construct the table:

1. Compute **FIRST set**
2. Compute **FOLLOW set**
3. For each production $A \rightarrow \alpha$, for each terminal in $\text{FIRST}(\alpha)$, add $A \rightarrow \alpha$ to table
4. If $\epsilon \in \text{FIRST}(\alpha)$, add $A \rightarrow \alpha$ for terminals in **FOLLOW(A)**
5. Blank entries in the table represent **errors**

■ These topics together explain how a compiler **checks program syntax and builds parse trees**. Useful for KTU Compiler Design exams.